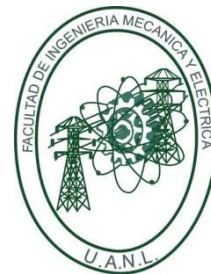




Universidad Autónoma de Nuevo León
Facultad de Ingeniería Mecánica y Eléctrica



*Controladores y Microcontroladores
Programables*

AFU03 - Producto Integrador de aprendizaje (PIA)

N.º de Equipo: 4

Brigada: 001

Nombre:	Matricula:	Numero de lista:
Heidi Pamela Martínez Martínez	1952947	6
Steven Neftalí Verdugo Figueroa	2132366	60
Ángel Armando Solís Jiménez	2132088	55
Carlos Emiliano Mendoza Ramírez	2048186	35

Nombre del profesor: Jesús Daniel Garza Camarena

Semestre Agosto – Diciembre 2025

Días de la clase y Hora: JueM4M6

San Nicolás de los Garza, N.L.

Fecha: 25/11/2025

Sistema de Control Lumínico Inteligente para Invernadero

1. REDACCIÓN DE LA ACTIVIDAD:

En este proyecto se solicita diseñar, simular y construir un prototipo funcional de un Sistema de Control Lumínico Inteligente para Invernadero, basado en un microcontrolador AVR ATmega328P programado en lenguaje C o ASM (sin uso de código o librerías de Arduino). El sistema tiene como finalidad regular actuadores de iluminación de un entorno de cultivo interior a pequeña escala, respondiendo dinámicamente a las condiciones de luz ambiente.

El sistema debe implementar una máquina de estados finitos que gestione tres estados operativos (IDLE, START y HOLD) de forma lógica y estructurada.

Durante el estado IDLE, el sistema presentará un mensaje de bienvenida propuesto por el usuario en el display de 4 dígitos de 7 segmentos multiplexado y estará en espera del botón de inicio.

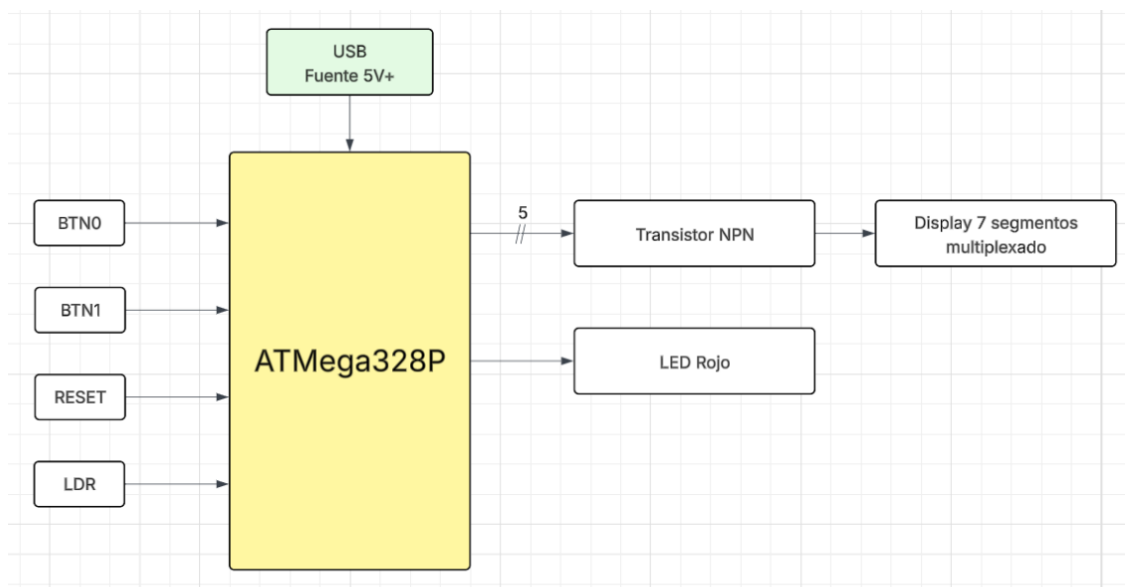
En el estado START, el sistema debe realizar la lectura de un sensor analógico mediante el ADC, y utilizar dicho valor para controlar proporcionalmente:

- El brillo de un LED representando la luz artificial.

Ambos elementos deben controlarse mediante PWM generado desde el microcontrolador. Los resultados del sensor deben mostrarse en una escala ajustada de 0 a 100 en el display.

En el estado HOLD, el valor del sensor se mantiene estático y el sistema entra en pausa operativa, indicando dicho estado mediante parpadeo del LED.

2. DIAGRAMA DE BLOQUES:



3. LISTA DE MATERIALES:

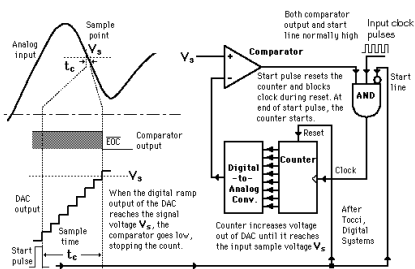
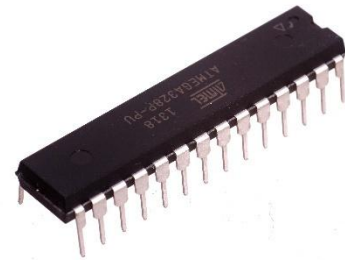
Material	Unidades	Especificación Técnica	Función en el Sistema	Precio Unitario (MXN)	Subtotal
Arduino Uno (ATmega328P)	1	Microcontrolador ATmega328P, 14 pines digitales, 6 entradas analógicas, funciona a 5V, USB y alimentación externa.	Actúa como el cerebro del sistema, ejecuta la lógica principal, ADC, PWM y multiplexado.	\$150	\$150
Protoboard	1	Tablero de pruebas sin soldadura, compatible con componentes electrónicos estándar.	Permite montar y probar el circuito sin soldadura.	\$40	\$40
Resistencia 1k Ω	4	Tolerancia 5%, ¼ W.	Pull-down o acondicionamiento de entradas digitales.	\$1	\$4
Resistencia 10k Ω	1	Tolerancia 5%, ¼ W.	Limitan corriente para el fotoresistor.	\$9	\$9
Resistencia 220 Ω	11	Tolerancia 5%, ¼ W.	Limitan corriente de segmentos del display y LED.	\$1	\$9
Display de 7 segmentos (4 dígitos)	1	Cátodo común / Ánodo común, tensión de operación 5 V.	Muestra los valores de 0 a 100 mediante multiplexación.	\$70	\$70
Jumpers	19	Cables de conexión macho-macho / hembra-macho / hembra-hembra.	Permiten realizar todas las conexiones del sistema.	\$1	\$19
Transistores NPN (BC547-B)	4	Ganancia hFE $\approx$ 100, corriente máx. 100 mA.	Activan cada dígito del display y proporcionan control de corriente.	\$3	\$12

Botones (push button)	3	Tensión 5 V, rebote mecánico ~10 ms.	Controlan las funciones START, HOLD y RESET.	\$3	\$9
Fotoresistor (LDR)	1	GL5528 / GL5537, resistencia variable según la luz.	Detecta la iluminación ambiente para regular el LED mediante ADC.	\$10	\$10
LED	1	LED 5 mm, 2–3 V, 20 mA.	Actuador regulado por PWM que simula la iluminación del invernadero.	\$2	\$2

4. MARCO TEÓRICO:

ATmega328P:

El ATmega328P es un microcontrolador AVR de 8 bits con 32 KB de memoria Flash, 1 KB de EEPROM y 2 KB de SRAM. Su arquitectura Harvard permite ejecutar instrucciones y acceder a memoria simultáneamente. Posee periféricos integrados como ADC, Timers, PWM y puertos I/O digitales.



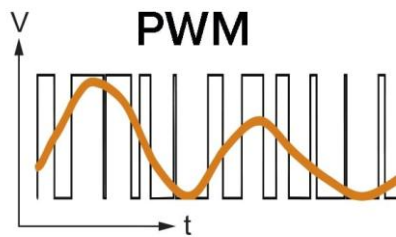
Conversión ADC:

El ADC del ATmega328P es de 10 bits y permite convertir señales analógicas en valores digitales entre 0–1023. En este proyecto se utiliza para medir la variación de voltaje proporcionada por un sensor LDR.

Sensores LDR:

Un LDR (Light Dependent Resistor) es un sensor resistivo cuya resistencia disminuye conforme aumenta la iluminación. Suelen emplearse en divisores de tensión. La salida se convierte mediante el ADC.



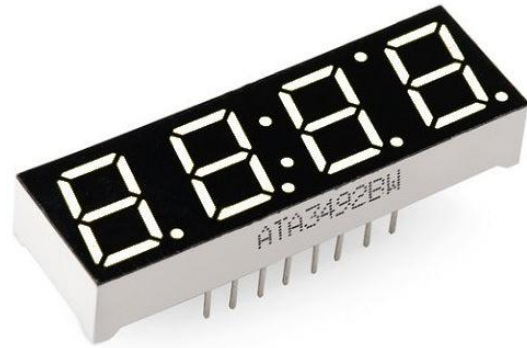


PWM:

La modulación por ancho de pulso permite controlar dispositivos mediante pulsos de alta frecuencia. El duty cycle ajusta el brillo del LED proporcional a la lectura del sensor.

Displays de 7 segmentos:

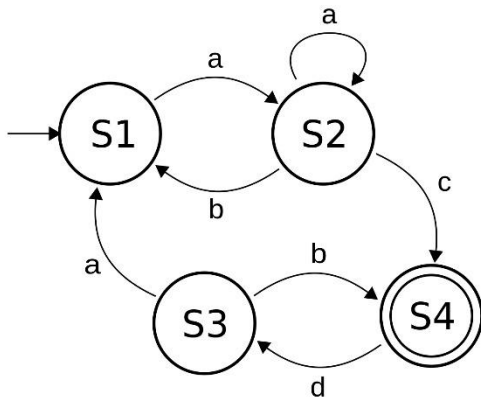
Un display de 7 segmentos consiste en 7 LEDs internos y uno para el punto decimal. El multiplexado permite controlar múltiples dígitos activándolos de manera alternada a alta velocidad.



Máquina de Estados Finitos:

Estado IDLE

El microcontrolador inicia en el estado IDLE al encenderse el sistema. Este estado establece las condiciones iniciales y proporciona al usuario un mensaje visual en el display de cuatro dígitos, indicando que el sistema está listo para comenzar. El mensaje de bienvenida se muestra durante unos segundos y luego cambia a un indicador de disponibilidad.



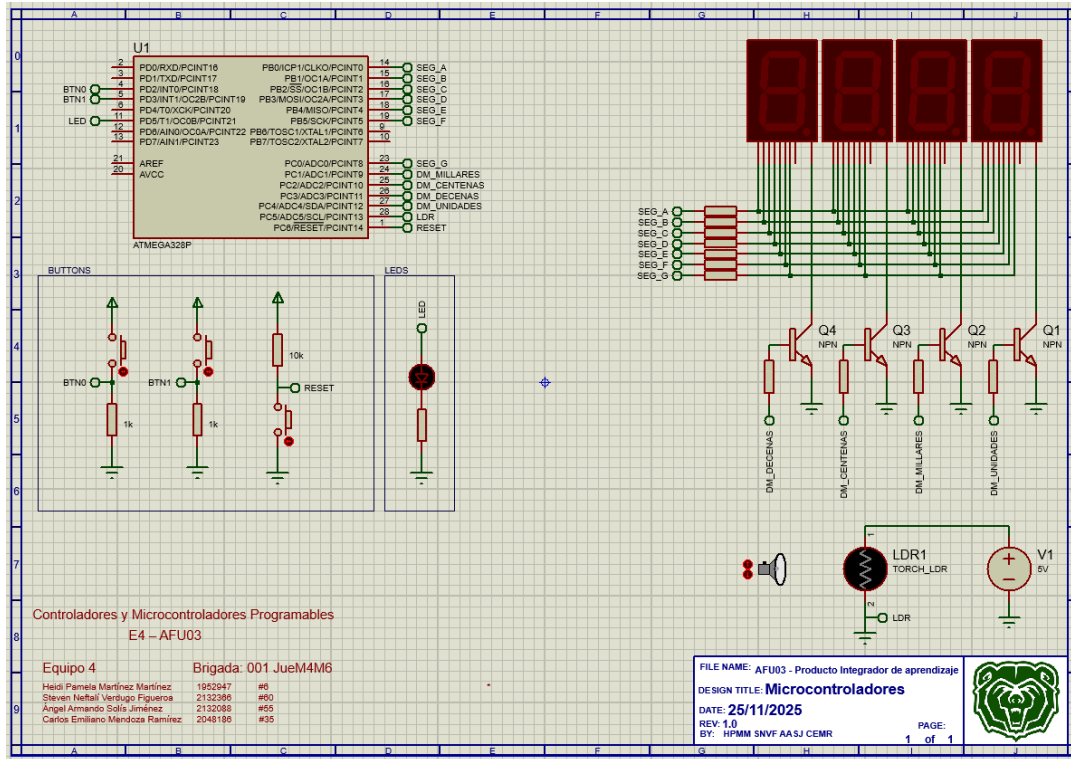
Durante este estado, el LED se mantiene encendido al cien por ciento mediante una señal de modulación por ancho de pulso. Esto representa una referencia luminosa constante que indica que el sistema está encendido y en espera. El microcontrolador permanece verificando el botón START. Mientras el usuario no presione este botón, el sistema no realiza lecturas del sensor ni control de brillo, manteniéndose en una espera controlada y estable.

La transición al estado START ocurre únicamente cuando el usuario presiona el botón correspondiente. Este comportamiento garantiza que el usuario tenga control total sobre el inicio del proceso de monitoreo y regulación.

Una FSM define el comportamiento del sistema según condiciones externas y transiciones lógicas. En este proyecto se usa para coordinar IDLE, START y HOLD.

5. DIAGRAMA ESQUEMÁTICO EN EDA

ESQUEMÁTICO EN PROTEUS



6. FUNDAMENTO DEL COMPORTAMIENTO

El funcionamiento del Sistema de Control Lumínico Inteligente se basa en la implementación de una Máquina de Estados Finitos (FSM), la cual organiza y determina las acciones del sistema según las entradas disponibles (botones y sensor LDR) y el estado actual del microcontrolador. Esta estructura permite que el comportamiento sea ordenado, predecible y escalable, lo que garantiza que el sistema responda correctamente a la interacción del usuario y a las variaciones de luz ambiental detectadas por el sensor. La FSM utilizada está compuesta por tres estados principales: IDLE, START y HOLD. Cada estado representa una etapa operativa diferente, con funciones bien definidas y transiciones explícitas basadas en eventos externos. La ejecución de estos estados se soporta mediante periféricos internos del ATmega328P, como el ADC de 10 bits, los temporizadores con generación PWM, el sistema de interrupciones y los puertos digitales configurados para multiplexar el display de siete segmentos.

Estado IDLE

El microcontrolador inicia en el estado IDLE al encenderse el sistema. Este estado establece las condiciones iniciales y proporciona al usuario un mensaje visual en el display de cuatro dígitos, indicando que el sistema está listo para comenzar. El

mensaje de bienvenida se muestra durante unos segundos y luego cambia a un indicador de disponibilidad.

Durante este estado, el LED se mantiene encendido al cien por ciento mediante una señal de modulación por ancho de pulso. Esto representa una referencia luminosa constante que indica que el sistema está encendido y en espera. El microcontrolador permanece verificando el botón START. Mientras el usuario no presione este botón, el sistema no realiza lecturas del sensor ni control de brillo, manteniéndose en una espera controlada y estable. La transición al estado START ocurre únicamente cuando el usuario presiona el botón correspondiente. Este comportamiento garantiza que el usuario tenga control total sobre el inicio del proceso de monitoreo y regulación.

Estado START

Al ingresar al estado START comienza el funcionamiento principal del sistema. El microcontrolador habilita el convertidor analógico digital para leer el valor proveniente del sensor LDR a través de un divisor de tensión. El valor leído se convierte a un número digital comprendido entre cero y mil veintitrés y posteriormente se transforma en un porcentaje de cero a cien para facilitar su visualización y control.

Con este porcentaje se ajusta el brillo del LED mediante una señal de modulación por ancho de pulso generada por uno de los temporizadores internos del microcontrolador. La intensidad del LED se adapta automáticamente según la luz ambiental detectada por el sensor. Cuando el ambiente es muy iluminado el LED reduce su brillo, y cuando el ambiente es oscuro el LED aumenta su intensidad. Esto simula el comportamiento de un sistema de iluminación inteligente capaz de compensar las variaciones de luz.

Estado HOLD

El estado HOLD permite conservar el último valor leído del sensor sin actualizarlo. El microcontrolador deja de realizar conversiones analógicas digitales y mantiene el último valor almacenado. Este valor se intercala visualmente con la palabra HOLD en el display, alternando cada segundo. De esta manera el usuario puede identificar que el sistema se encuentra en pausa operativa.

En lugar de mantener el brillo controlado por la luz del ambiente, el LED entra en un modo de parpadeo constante que sirve como indicador visual de pausa. Este parpadeo se genera mediante temporización controlada por software o utilizando los temporizadores internos.

Interpretación general

La Máquina de Estados organiza de manera clara las funciones del sistema y define cómo deben comportarse las entradas y salidas en cada etapa de operación. Esta división facilita la comprensión del sistema, su mantenimiento y la posible expansión o mejora del proyecto. La integración del sensor con el convertidor analógico digital, el control del LED mediante modulación por ancho de pulso y la visualización mediante multiplexación permiten construir un sistema sólido y funcional que responde adecuadamente a los requerimientos del proyecto.

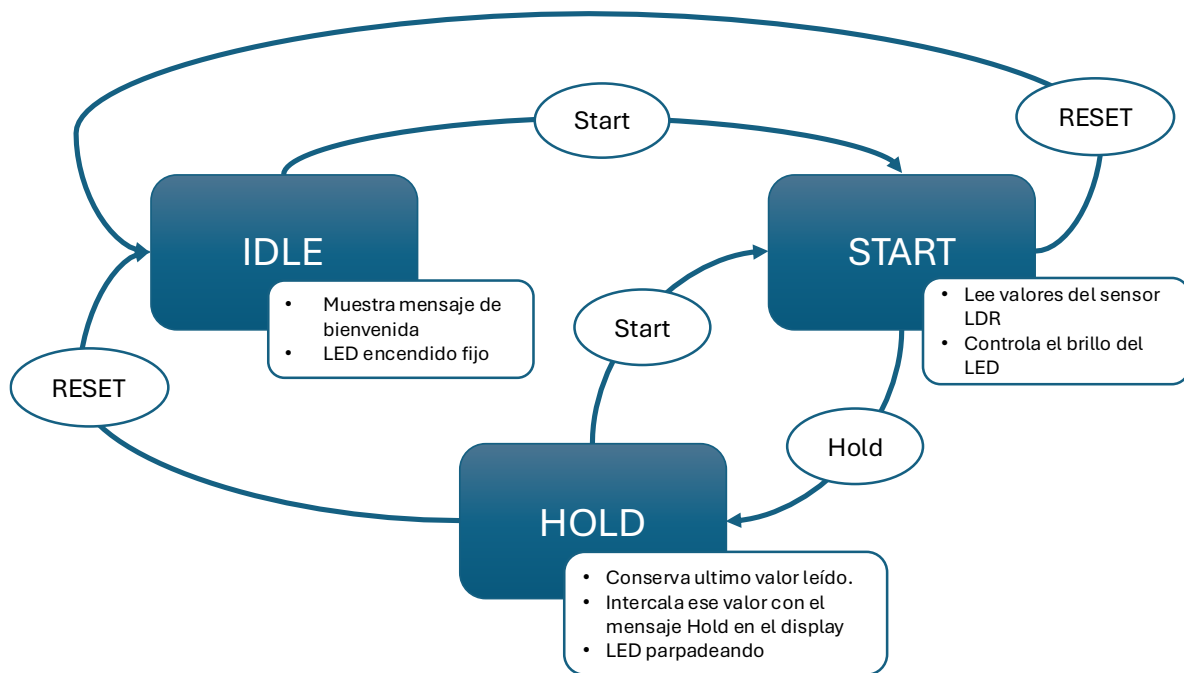
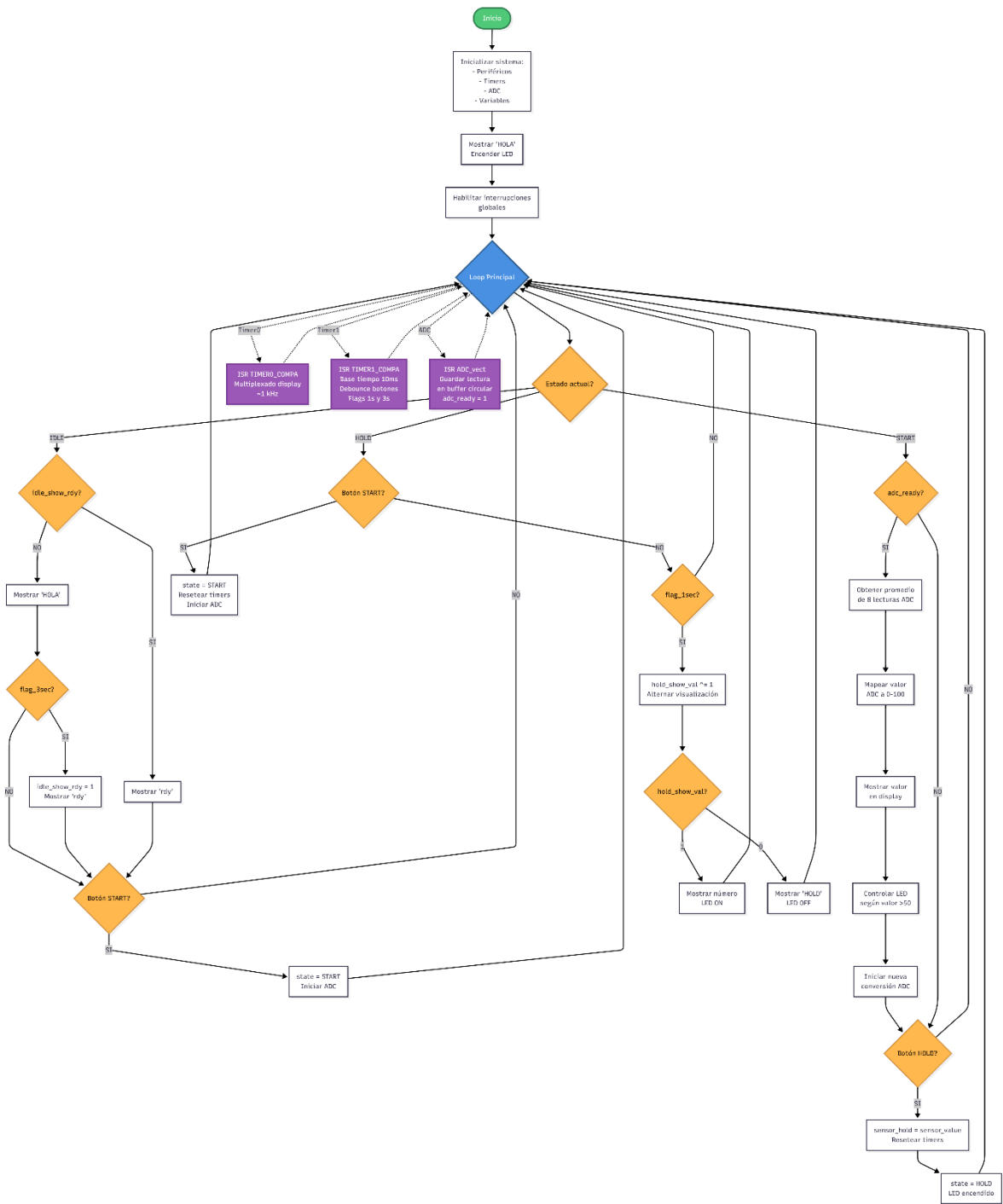


Diagrama de flujo



7. TABLA DE PINES:

Pin ATmega328P	Arduino	Tipo	Función	Elemento conectado
PB0	D8	Salida	Segmento A del display	Display 7 segmentos (segmento A)
PB1	D9	Salida	Segmento B del display	Display 7 segmentos (segmento B)
PB2	D10	Salida	Segmento C del display	Display 7 segmentos (segmento C)
PB3	D11	Salida	Segmento D del display	Display 7 segmentos (segmento D)
PB4	D12	Salida	Segmento E del display	Display 7 segmentos (segmento E)
PB5	D13	Salida	Segmento F del display	Display 7 segmentos (segmento F)
PC0	A0	Salida	Segmento G del display	Display 7 segmentos (segmento G)
PC1	A1	Salida	Control dígito 1 (multiplexado)	Display 7 segmentos (dígito 1)
PC2	A2	Salida	Control dígito 2 (multiplexado)	Display 7 segmentos (dígito 2)
PC3	A3	Salida	Control dígito 3 (multiplexado)	Display 7 segmentos (dígito 3)
PC4	A4	Salida	Control dígito 4 (multiplexado)	Display 7 segmentos (dígito 4)
PC5	A5	Entrada analógica	Canal ADC5 - Sensor de luz	Sensor de luz (fotoresistencia/LDR)
PC6	RESET	Entrada digital	Botón RESET (pull-up interno)	Pulsador RESET
PD2	D2	Entrada digital	Botón START (pull-up interno)	Pulsador START
PD3	D3	Entrada digital	Botón HOLD (pull-up interno)	Pulsador HOLD
PD5	D4	Salida	Control de LED indicador	LED de estado

La tabla de pines muestra cómo se asignan las entradas y salidas del microcontrolador ATmega328P para permitir el funcionamiento del sistema. Cada pin cumple una función específica dentro del control del sensor, los botones, el display y el LED.

El Puerto B se encarga de manejar seis de los siete segmentos del display de 7 segmentos, específicamente los segmentos A, B, C, D, E y F. Estos segmentos están conectados a los pines PB0 hasta PB5, que en la nomenclatura de Arduino corresponden a los pines digitales D8 hasta D13. Todos estos pines están

configurados como salidas digitales para enviar las señales que forman los números y letras en el display.

El Puerto C tiene una función mixta muy importante en el proyecto. El pin PC0 (equivalente a A0 en Arduino) controla el segmento G del display de 7 segmentos, completando así los siete segmentos necesarios. Los siguientes cuatro pines, PC1 a PC4 (A1 a A4 en Arduino), controlan el multiplexado del display, activando cada uno de los cuatro dígitos de manera secuencial. Esta técnica permite mostrar cuatro números simultáneamente, aunque en realidad se encienden uno a la vez a alta velocidad. Finalmente, el pin PC5 (A5 en Arduino) cumple una función completamente diferente: es una entrada analógica conectada al sensor de luz (fotoresistencia o LDR), utilizando el canal 5 del conversor analógico-digital (ADC) para leer los valores de iluminación del ambiente.

El Puerto D maneja la interacción con el usuario y la señalización visual. Los pines PD2 y PD3 (D2 y D3 en Arduino) están configurados como entradas digitales con resistencias pull-up internas habilitadas, lo que significa que están conectados a los botones START y HOLD respectivamente. Estos botones se conectan entre el pin correspondiente y tierra (GND), de modo que cuando se presionan, el pin lee un nivel bajo. El pin PD5 (D5 en Arduino) es una salida digital que controla el LED indicador del sistema, el cual cambia su estado dependiendo del modo de operación: permanece encendido en modo IDLE, se modula según el nivel de luz detectado en modo START, y parpadea cada segundo en modo HOLD.

Además de estos pines específicos, el circuito requiere las conexiones de alimentación estándar: VCC y GND para proporcionar los +5V y tierra al microcontrolador, AVCC que sirve como voltaje de referencia para el conversor analógico-digital (también conectado a +5V con un capacitor de desacople para filtrar el ruido), y opcionalmente AREF que en este caso se deja sin conectar ya que se utiliza AVCC como referencia para las mediciones analógicas. Esta distribución de pines permite que el sistema funcione de manera eficiente, utilizando un total de 15 pines del ATmega328P para controlar el display multiplexado de 4 dígitos, leer el sensor de luz, procesar las entradas de dos botones y controlar un LED indicador.

8. CÓDIGO EN LENGUAJE C O ASM NO ARDUINO

```

/*****
*   Copyright (C) 2025 by Equipo 4
*   Heidi Pamela Martínez Martínez    1952947    #6
*   Steven Neftalí Verdugo Figueroa    2132366    #60
*   Ángel Armando Solís Jiménez        2132088    #55
*   Carlos Emiliano Mendoza Ramírez    2048186    #35
*
*   Attribution-NonCommercial-ShareAlike 3.0(CC-BY-NC-SA 3.0)
*   AFU03 - Producto Integrador de aprendizaje (PIA)
*   Dispositivo: atmega328p
*   Lenguaje C
*   Rev: 1.1
*   Fecha: 25/11/25
*****/
```

```

*****/

#include <avr/io.h>
#define F_CPU 16000000UL // Frecuencia de reloj del microcontrolador 16 MHz
#include <util/delay.h>
#include <avr/interrupt.h>

//===== DISPLAY DE 7 SEGMENTOS =====//
// Configuración de los puertos para los segmentos A-F del display 7 segmentos
#define DISPLAY_7SEG_AF_DDRX DDRB // Dirección: puerto B para segmentos A-F
#define DISPLAY_7SEG_AF_PORTX PORTB // Puerto de salida para segmentos A-F
#define DISPLAY_7SEG_G_DDRX DDRC // Dirección: puerto C para segmento G
#define DISPLAY_7SEG_G_PORTX PORTC // Puerto de salida para segmento G
#define DISPLAY_7SEG_SEGG PORTC0 // Segmento G en PC0

//===== BOTONES =====//
#define BTN_START_PIN PORTD2 // Botón START en D2
#define BTN_HOLD_PIN PORTD3 // Botón HOLD en D3

//===== DISPLAY MULTIPLEXADO =====//
// Configuración para el multiplexado de 4 dígitos en el display
#define DM_DDRX DDRC
#define DM_PORTX PORTC
#define DM_DIG1 PORTC1 // Dígito 1
#define DM_DIG2 PORTC2 // Dígito 2
#define DM_DIG3 PORTC3 // Dígito 3
#define DM_DIG4 PORTC4 // Dígito 4

//===== LED Y SENSOR =====//
#define LED_DDRX DDRD // Dirección del LED
#define LED_PORTX PORTD // Puerto del LED
#define LED_PIN PORTD5 // LED en D5

#define SENSOR_ADC_CHANNEL 5 // Sensor de luz en A5 (canal 5 del ADC)

//===== VARIABLES GLOBALES =====//
// Array para almacenar los 4 dígitos a mostrar en el display
volatile uint8_t display_digits[4] = {0, 0, 0, 0};

// Patrones binarios para mostrar números 0-9 en el display 7 segmentos
uint8_t display_numbers[10] = {
    0b00111111, 0b00000110, 0b01011011, 0b01001111, 0b01100110,
    0b01101101, 0b01111101, 0b00000111, 0b01111111, 0b01101111
};

// Patrones binarios para mostrar letras en el display 7 segmentos
const uint8_t LETTER_H = 0b01110110; // Letra H
const uint8_t LETTER_O = 0b00111111; // Letra O
const uint8_t LETTER_L = 0b00111000; // Letra L
const uint8_t LETTER_A = 0b01110111; // Letra A
const uint8_t LETTER_d = 0b01011110; // Letra d
const uint8_t LETTER_r = 0b01010000; // Letra r
const uint8_t LETTER_y = 0b01101110; // Letra y
const uint8_t LETTER_SPACE = 0b00000000; // Espacio en blanco

// Variables para los timers
volatile uint8_t timer0_multiplex = 0; // Contador para multiplexado del display
volatile uint16_t timer1_1sec = 0; // Contador para bandera de 1 segundo

```

```

volatile uint8_t flag_1sec      = 0;    // Bandera que se activa cada 1 segundo
volatile uint16_t timer1_3sec   = 0;    // Contador para bandera de 3 segundos
volatile uint8_t flag_3sec      = 0;    // Bandera que se activa cada 3 segundos

// Variables para los botones
volatile uint8_t btn_start = 0;          // Bandera del botón START
volatile uint8_t btn_hold  = 0;          // Bandera del botón HOLD

// Variables para debounce (anti-rebote de botones)
volatile uint8_t hold_debounce_count = 0; // Contador debounce para HOLD
volatile uint8_t hold_pressed        = 0; // Estado presionado de HOLD
volatile uint8_t start_debounce_count = 0; // Contador debounce para START
volatile uint8_t start_pressed        = 0; // Estado presionado de START

// Variables del ADC (sensor de luz)
volatile uint16_t adc_value  = 0;        // Valor bruto del ADC
volatile uint8_t  adc_ready  = 0;        // Bandera de conversión completada
volatile uint8_t  sensor_value = 0;      // Valor mapeado (0-100) del sensor
volatile uint8_t  sensor_hold = 0;      // Valor guardado en HOLD

// Buffer para promediar lecturas del ADC (filtra ruido)
// Se inicializa con 1 para evitar división por cero
volatile uint16_t adc_buffer[8] = {1, 1, 1, 1, 1, 1, 1, 1};
volatile uint8_t  adc_buffer_idx = 0;    // Índice actual del buffer

// Estados del sistema
enum states { STATE_IDLE, STATE_START, STATE_HOLD };
volatile enum states state;              // Estado actual del sistema

volatile uint8_t idle_show_rdy = 0;      // Bandera para mostrar "rdy" en IDLE
volatile uint8_t hold_show_val = 1;      // Bandera para alternar visualización en
HOLD

//===== PROTOTIPOS =====//
void system_init(void);                 // Inicializa todos los periféricos
void display_show_segment(uint8_t code); // Muestra un patrón en el display
void display_show_number(uint16_t num);  // Muestra un número en el display
void display_show_hola(void);            // Muestra "HOLA" en el display
void display_show_rdy(void);             // Muestra "rdy" en el display
void display_show_hold(void);            // Muestra "HOLD" en el display
void display_multiplex(void);            // Multiplexado del display
void led_set_brightness(uint8_t bright); // Controla el brillo del LED
uint8_t map_sensor(uint16_t adc);        // Mapea valor ADC a 0-100
uint16_t get_adc_average(void);          // Calcula promedio del buffer

//===== MAIN =====//
int main(void)
{
    cli();                             // Desabilita interrupciones globales
    system_init();                      // Inicializa periféricos

    // Inicializa todas las variables
    state      = STATE_IDLE;
    idle_show_rdy = 0;
    timer1_3sec = 0;
    flag_3sec   = 0;
    timer1_1sec = 0;
    flag_1sec   = 0;

```

```

btn_start      = 0;
btn_hold       = 0;
hold_debounce_count = 0;
hold_pressed    = 0;
start_debounce_count = 0;
start_pressed    = 0;
hold_show_val   = 1;

display_show_hola();           // Muestra "HOLA" en el display
LED_PORTX |= (1<<LED_PIN);    // Enciende el LED al iniciar

sei();                        // Habilita interrupciones globales

_delay_ms(200);               // Pequeño retardo de inicialización

while (1)                    // Loop principal
{
    switch (state)
    {
        case STATE_IDLE:
            // Estado de reposo/inicio
            if (idle_show_rdy == 0)
            {
                display_show_hola();

                // Después de 3 segundos, cambia a mostrar "rdy"
                if (flag_3sec == 1)
                {
                    idle_show_rdy = 1;
                    flag_3sec      = 0;
                    timer1_3sec     = 0;
                    display_show_rdy();
                }
            }
            else
            {
                display_show_rdy();
            }

            LED_PORTX |= (1<<LED_PIN); // LED encendido en IDLE

            // Si presionan START, inicia la lectura del sensor
            if (btn_start == 1)
            {
                btn_start = 0;
                state = STATE_START;
                ADCSRA |= (1<<ADSC); // Inicia conversión ADC
            }
            break;

        case STATE_START:
            // Estado de lectura del sensor
            if (adc_ready == 1)
            {
                adc_ready = 0;

                // Obtiene promedio de 8 lecturas (filtra ruido)
                uint16_t adc_avg = get_adc_average();
            }
        }
    }
}

```

```

        // Mapea el valor promedio a 0-100
        sensor_value = map_sensor(adc_avg);

        // Muestra el valor en el display (0-100)
        display_show_number(sensor_value);

        // Controla el LED según el valor del sensor
        led_set_brightness(sensor_value);

        // Inicia nueva conversión ADC
        ADCSRA |= (1<<ADSC);
    }

    // Si presionan HOLD, guarda el valor actual
    if (btn_hold == 1)
    {
        btn_hold      = 0;
        sensor_hold    = sensor_value; // Guarda el valor del sensor
        hold_show_val  = 1;
        timer1_1sec    = 0;
        flag_1sec      = 0;

        display_show_number(sensor_hold);
        LED_PORTX |= (1<<LED_PIN); // Enciende LED al entrar a HOLD
        state = STATE_HOLD;        // Entra a estado HOLD
    }
    break;

case STATE_HOLD:
    // Estado de pausa - el LED parpadea cada segundo
    if (btn_start == 1)
    {
        // Si presionan START, regresa a leer el sensor
        btn_start = 0;

        timer1_1sec    = 0;
        flag_1sec      = 0;
        hold_show_val  = 1;

        state = STATE_START;
        ADCSRA |= (1<<ADSC);
        break;
    }

    // Cada 1 segundo, alterna la visualización del LED
    if (flag_1sec == 1)
    {
        flag_1sec = 0;
        hold_show_val ^= 1; // Alterna entre 0 y 1

        if (hold_show_val)
        {
            // Muestra el valor y enciende el LED (siempre, sin importar
el valor)

            display_show_number(sensor_hold);
            LED_PORTX |= (1<<LED_PIN);
        }
    }

```

```

        else
        {
            // Muestra "HOLD" y apaga el LED (efecto parpadeo)
            display_show_hold();
            LED_PORTX &= ~(1<<LED_PIN);
        }
    }
    break;
}
}

//===== IMPLEMENTACIÓN =====//

void system_init(void)
{
    // Configura el display de 7 segmentos como salidas
    DISPLAY_7SEG_AF_DDRX |= 0x3F;           // Segmentos A-F en puerto B
    DISPLAY_7SEG_G_DDRX  |= (1<<DISPLAY_7SEG_SEGG); // Segmento G en puerto C

    // Configura los dígitos multiplexados como salidas
    DM_DDRX |= (1<<DM_DIG1) | (1<<DM_DIG2) | (1<<DM_DIG3) | (1<<DM_DIG4);

    // Configura los botones como entradas con pull-up interno
    DDRD &= ~(1<<BTN_START_PIN); // D2 como entrada
    PORTD |= (1<<BTN_START_PIN); // Habilita pull-up en D2

    DDRD &= ~(1<<BTN_HOLD_PIN); // D3 como entrada
    PORTD |= (1<<BTN_HOLD_PIN); // Habilita pull-up en D3

    // Configura el LED como salida
    LED_DDRX |= (1<<LED_PIN); // D5 como salida
    LED_PORTX |= (1<<LED_PIN); // LED encendido desde el inicio

    // Configura el ADC (convertor analógico-digital)
    ADMUX = (1<<REFS0) | (SENSOR_ADC_CHANNEL & 0x0F); // Referencia AVCC, canal 5
    ADCSRA = (1<<ADEN)|(1<<ADIE)|(1<<ADPS2)|(1<<ADPS1)|(1<<ADPS0); // Habilita ADC
    con interrupción

    // Configura Timer0: para multiplexado del display (velocidad ~1 kHz)
    TCCR0A = (1<<WGM01); // Modo CTC
    OCR0A = 156; // Comparador para frecuencia
    TIMSK0 = (1<<OCIE0A); // Habilita interrupción de comparación
    TCCR0B = (1<<CS02)|(1<<CS00); // Prescaler 1024

    // Configura Timer1: base de 10ms para generar flags de 1s y 3s
    TCCR1B = (1<<WGM12)|(1<<CS11)|(1<<CS10); // Modo CTC, prescaler 64
    OCR1A = 2500; // Genera interrupción cada 10ms
    TIMSK1 = (1<<OCIE1A); // Habilita interrupción de comparación
}

void display_show_segment(uint8_t code)
{
    // Muestra un patrón de 7 segmentos en el display
    // Los primeros 6 bits van al puerto B (segmentos A-F)
    // El bit 6 va al puerto C (segmento G)
    DISPLAY_7SEG_AF_PORTX = (code & 0x3F) | (DISPLAY_7SEG_AF_PORTX & 0xC0);
    DISPLAY_7SEG_G_PORTX = ((code>>6) & 0x01) | (DISPLAY_7SEG_G_PORTX & 0xFE);
}

```

```

}

void display_show_number(uint16_t num)
{
    // Convierte un número a sus 4 dígitos individuales
    if (num > 9999) num = 9999; // Limita a 4 dígitos
    display_digits[0] = (num / 1000) % 10; // Miles
    display_digits[1] = (num / 100) % 10; // Centenas
    display_digits[2] = (num / 10) % 10; // Decenas
    display_digits[3] = num % 10; // Unidades
}

void display_show_hola(void)
{
    // Muestra la palabra "HOLA" en el display (saludo inicial)
    display_digits[0] = 10; // H
    display_digits[1] = 11; // O
    display_digits[2] = 12; // L
    display_digits[3] = 13; // A
}

void display_show_rdy(void)
{
    // Muestra "rdy" en el display (listo/ready)
    display_digits[0] = 14; // espacio
    display_digits[1] = 15; // r
    display_digits[2] = 16; // d
    display_digits[3] = 17; // y
}

void display_show_hold(void)
{
    // Muestra "HOLD" en el display (pausa)
    display_digits[0] = 10; // H
    display_digits[1] = 11; // O
    display_digits[2] = 12; // L
    display_digits[3] = 16; // d
}

void display_multiplex(void)
{
    // Función llamada por Timer0 para mostrar cada dígito secuencialmente
    // Esto crea el efecto de ver los 4 dígitos simultáneamente

    uint8_t code;
    uint8_t val = display_digits[timer0_multiplex]; // Obtiene el dígito actual

    // Busca el patrón binario para el dígito o letra
    if (val < 10)
        code = display_numbers[val]; // Es un número
    else
    {
        // Es una letra o símbolo especial
        switch(val)
        {
            case 10: code = LETTER_H; break;
            case 11: code = LETTER_O; break;
            case 12: code = LETTER_L; break;
        }
    }
}

```

```

        case 13: code = LETTER_A;      break;
        case 14: code = LETTER_SPACE; break;
        case 15: code = LETTER_r;      break;
        case 16: code = LETTER_d;      break;
        case 17: code = LETTER_y;      break;
        default: code = 0;              break;
    }
}

// Apaga todos los dígitos
DM_PORTX &= 0xE1;

// Enciende el dígito actual
switch (timer0_multiplex)
{
    case 0: DM_PORTX |= (1<<DM_DIG1); break;
    case 1: DM_PORTX |= (1<<DM_DIG2); break;
    case 2: DM_PORTX |= (1<<DM_DIG3); break;
    case 3: DM_PORTX |= (1<<DM_DIG4); break;
}

// Muestra el patrón en el display
display_show_segment(code);
}

void led_set_brightness(uint8_t bright)
{
    // Controla el LED basado en el valor del sensor
    // Valores mayores a 50 encienden el LED (mucha luz detectada)
    if (bright > 50)
        LED_PORTX |= (1<<LED_PIN);    // LED encendido
    else
        LED_PORTX &= ~(1<<LED_PIN);    // LED apagado
}

// Obtiene promedio de 8 lecturas del ADC para filtrar ruido
uint16_t get_adc_average(void)
{
    uint32_t suma = 0;
    for (uint8_t i = 0; i < 8; i++)
    {
        suma += adc_buffer[i]; // Suma todos los valores del buffer
    }
    return (uint16_t)(suma / 8); // Retorna el promedio
}

uint8_t map_sensor(uint16_t adc)
{
    // Mapea el valor bruto del ADC (1-13) a un rango de 0-100
    // Rango mínimo: 1 = máxima luz (retorna 100)
    // Rango máximo: 13 = oscuridad total (retorna 0)

    const uint16_t ADC_MIN = 1;    // Valor con mucha luz
    const uint16_t ADC_MAX = 13;    // Valor sin luz (oscuro)

    // Protección contra valores fuera de rango
    if (adc <= ADC_MIN) return 100; // Mucha luz = valor máximo
    if (adc >= ADC_MAX) return 0;   // Oscuro = valor mínimo
}

```

```

    // Mapeo INVERTIDO: valores bajos de ADC dan valores altos (más luz)
    uint32_t range = ADC_MAX - ADC_MIN;
    uint32_t diff = ADC_MAX - adc;
    uint32_t result = (diff * 100) / range;

    return (result > 100) ? 100 : (uint8_t)result;
}

//===== ISR (Rutinas de Interrupción) =====//

ISR(TIMER0_COMPA_vect)
{
    // Interrupción del Timer0: se ejecuta ~1000 veces por segundo
    // Controla el multiplexado del display

    display_multiplex();                // Muestra el dígito actual
    timer0_multiplex++;                 // Avanza al siguiente dígito
    if (timer0_multiplex >= 4)          // Si completó los 4 dígitos
        timer0_multiplex = 0;          // Vuelve al primero
}

ISR(TIMER1_COMPA_vect)
{
    // Interrupción del Timer1: se ejecuta cada 10ms (base de tiempo)

    // Contador de 1 segundo (100 interrupciones × 10ms = 1 segundo)
    timer1_1sec++;
    if (timer1_1sec >= 100)
    {
        timer1_1sec = 0;
        flag_1sec = 1; // Activa bandera de 1 segundo
    }

    // Contador de 3 segundos solo cuando está en IDLE y no muestra "rdy"
    if (state == STATE_IDLE && idle_show_rdy == 0)
    {
        timer1_3sec++;
        if (timer1_3sec >= 300) // 300 × 10ms = 3 segundos
        {
            flag_3sec = 1; // Activa bandera de 3 segundos
        }
    }
    else
    {
        // Reinicia contadores si no cumple condición
        timer1_3sec = 0;
        flag_3sec = 0;
    }

    // ===== DEBOUNCE BOTÓN START (D2) =====
    // Lee el estado del botón y implementa anti-rebote
    uint8_t start_pin_state = (PIND & (1 << BTN_START_PIN)) == 0; // 0 = presionado

    if (start_pin_state) // Botón presionado (LOW)
    {
        // Incrementa contador de debounce
        if (start_debounce_count < 10)

```

```

        start_debounce_count++;

        // Después de 5 interrupciones (50ms de presión), confirma
        if (start_debounce_count >= 5 && !start_pressed)
        {
            start_pressed = 1; // Marca como presionado
            btn_start      = 1; // Activa bandera
        }
    }
else // Botón suelto (HIGH)
{
    // Reinicia contadores cuando se suelta el botón
    start_debounce_count = 0;
    start_pressed = 0;
}

// ===== DEBOUNCE BOTÓN HOLD (D3) =====
// Lee el estado del botón y implementa anti-rebote
uint8_t hold_pin_state = (PIND & (1 << BTN_HOLD_PIN)) == 0; // 0 = presionado

if (hold_pin_state) // Botón presionado (LOW)
{
    // Incrementa contador de debounce
    if (hold_debounce_count < 10)
        hold_debounce_count++;

    // Después de 5 interrupciones (50ms de presión), confirma
    if (hold_debounce_count >= 5 && !hold_pressed)
    {
        hold_pressed = 1; // Marca como presionado
        btn_hold      = 1; // Activa bandera
    }
}
else // Botón suelto (HIGH)
{
    // Reinicia contadores cuando se suelta el botón
    hold_debounce_count = 0;
    hold_pressed = 0;
}
}

ISR(ADC_vect)
{
    // Interrupción del conversor ADC: se ejecuta al completar una conversión

    // Lee el valor del ADC (10 bits, rango 0-1023)
    uint16_t val = ADC;

    // Solo guarda valores válidos (descarta 0 que indica contacto intermitente)
    if (val > 0)
    {
        // Almacena el valor en el buffer circular
        adc_buffer[adc_buffer_idx] = val;

        // Avanza al siguiente índice del buffer (circular)
        adc_buffer_idx = (adc_buffer_idx + 1) % 8;
    }
}

```

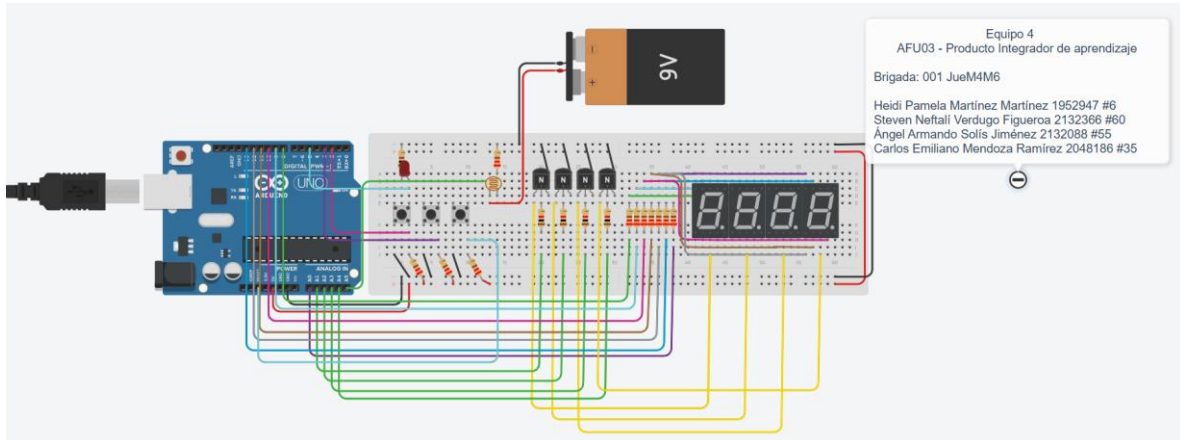
```

// Activa bandera indicando que hay nueva lectura disponible
adc_ready = 1;
}

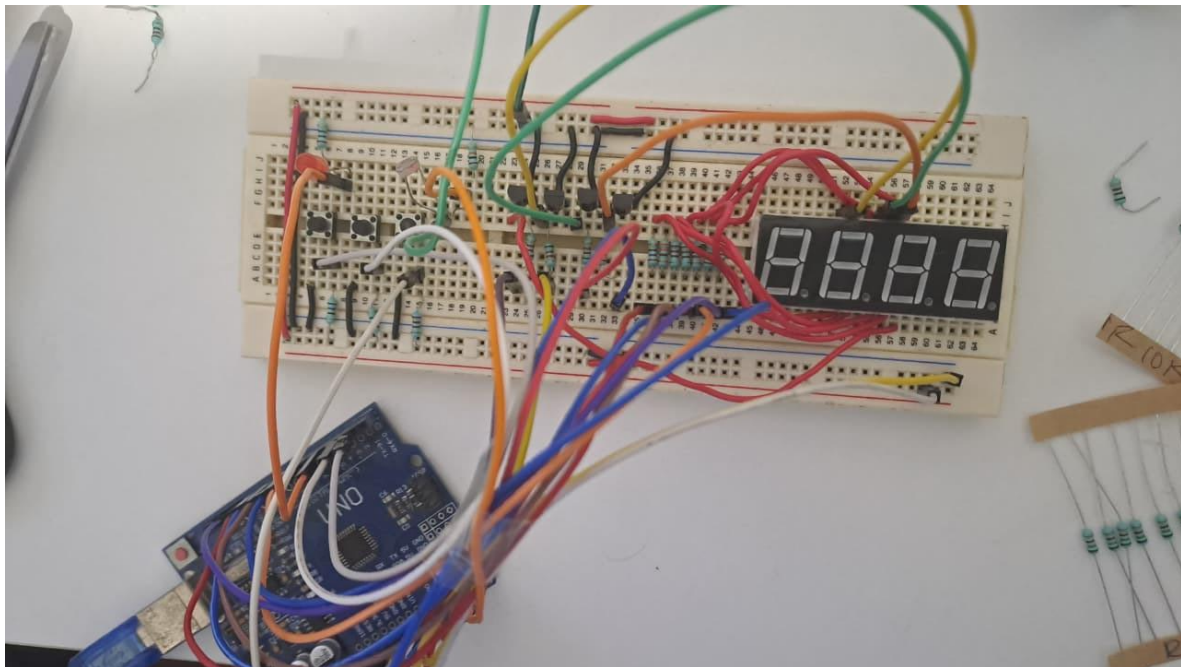
```

9. FOTOGRAFÍA DEL PROTOTIPO

ESQUEMÁTICO EN TINKERCAD



PROTOTIPO EN FÍSICO



CONCLUSIÓN PERSONAL

Heidi Pamela Martínez Martínez 1952947 #6

Durante este proyecto aprendí a programar el microcontrolador ATmega328P en lenguaje C a nivel de registros, trabajando directamente con los periféricos sin usar librerías de Arduino. Implementé una máquina de estados finitos (FSM) con tres estados (IDLE, START y HOLD) para organizar el flujo del programa, y utilicé interrupciones (ISRs) para manejar el multiplexado del display, la lectura del sensor ADC y el debounce de botones.

El principal problema que enfrenté fue en el estado HOLD: el LED no parpadeaba cuando el valor guardado era bajo porque la función `led_set_brightness()` lo apagaba. Lo resolví controlando el LED directamente en HOLD para garantizar el parpadeo independientemente del valor. Otro reto fue el ruido en las lecturas del sensor, que solucioné implementando un buffer circular de 8 posiciones para promediar los valores. También tuve que implementar debounce por software para evitar detecciones falsas en los botones.

Aprendí técnicas importantes como el multiplexado de displays, el uso eficiente de interrupciones, el filtrado de señales ruidosas y la calibración de sensores. Esta experiencia me enseñó que el desarrollo embebido requiere considerar tanto el software como las características físicas del hardware, y me dio las bases para enfrentar proyectos más complejos en el futuro.

Steven Neftalí Verdugo Figueroa 2132366 #60

Mi experiencia en este proyecto se centró en el análisis del comportamiento del sistema y en la importancia de la estabilidad en las lecturas del sensor. Implementar técnicas como el filtrado del ADC y el debounce me ayudó a comprender cómo pequeñas variaciones pueden afectar el rendimiento global si no se consideran adecuadamente. Trabajar con un microcontrolador a bajo nivel me permitió ver de manera detallada cómo cada configuración influye en el resultado final. Esta práctica fortaleció mi capacidad de diagnóstico y mi atención a los detalles, aspectos fundamentales en cualquier proyecto de electrónica.

Ángel Armando Solís Jiménez 2132088 #55

Pude reforzar la importancia de diseñar una Máquina de Estados bien definida, ya que esta estructura fue fundamental para organizar el comportamiento del sistema y evitar confusiones durante la ejecución. Entender cómo se sincronizan los tiempos, cómo se filtra el ruido del sensor y cómo se crean transiciones claras entre estados fue clave para lograr un funcionamiento estable. Además, trabajar en la optimización del código y depurar errores me enseñó que la paciencia, la atención al detalle y la documentación constante son esenciales para obtener resultados

profesionales. Me dejó claro que la electrónica y la programación se complementan perfectamente cuando se entienden desde su nivel fundamental. Considero que todo lo aprendido será un pilar importante para enfrentar retos más complejos en el futuro, tanto en el ámbito académico como profesional.

Carlos Emiliano Mendoza Ramírez 2048186 #35

Participar en este proyecto me permitió adquirir una visión más amplia sobre la construcción de un sistema embebido real y comprender cómo se integran tecnologías como sensores analógicos, displays multiplexados y control por PWM dentro de un mismo dispositivo. A lo largo del desarrollo, pude observar cómo decisiones aparentemente pequeñas como el manejo del rebote en los botones o la forma de promediar las lecturas del ADC pueden marcar la diferencia entre un sistema confiable y uno inestable. Este tipo de detalles, que normalmente pasan desapercibidos, fueron fundamentales para pulir el funcionamiento final del prototipo. La parte más valiosa para mí fue entender el flujo completo del procesamiento de datos: desde cómo el LDR varía su resistencia según la iluminación, hasta cómo esa lectura es interpretada digitalmente y convertida en un porcentaje que se ve reflejado tanto en el display como en el brillo del LED. Esto me enseñó a visualizar el sistema como un conjunto de etapas coordinadas, no como módulos aislados. Además, pude mejorar mis habilidades para estructurar código de manera ordenada, separar funciones, definir estados claros y analizar el comportamiento del firmware durante las pruebas físicas.

REFERENCIAS BIBLIOGRÁFICAS

Microchip Technology Inc. (2024). ATmega328P. <https://www.microchip.com/en-us/product/ATmega328P>

Robello, C. A. (S/F). Arquitectura del Microcontrolador ATMEGA328 y su programación utilizando Arduino. <http://robello.com.ar/comunes/arduinoplatformiovscode2022.pdf>

Industrias GSL. (2022). Sensor LDR. <https://industriasgsl.com/blogs/automatizacion/sensor-ldr>

LEDYi Lighting. (2023). Control LED eficiente: exploración de la atenuación PWM. <https://www.ledyilighting.com/es/led-pwm-dimming/>

Venero Lovaton, J. A. (S/F). Multiplexación de Display de 7 Segmentos. Scribd. <https://fr.scribd.com/document/646754869/Multiplexacion-de-Display-de-7-Segmentos>

Game Manual 0. (S/F). Máquinas de Estados Finitos. <https://gm0.org/es/latest/docs/software/concepts/finite-state-machines.html>